



Optimizing applications





Measuring: strace

- ▶ Allows to trace all the system calls made by an application and its children.
- ▶ Useful to:
 - Understand how time is spent in user space
 - For example, easy to find file open attempts (`open()`), file access (`read()`, `write()`), and memory allocations (`mmap2()`). Can be done without any access to source code!
 - Find the biggest time consumers (low hanging fruit)
 - Find unnecessary work done in applications and scripts. Example: opening the same file(s) multiple times, or trying to open files that do not exist.
- ▶ Limitation: you can't trace the `init` process!



Measuring: strace

System call tracer - <https://strace.io>

- ▶ Available on all GNU/Linux systems Can be built by your cross-compiling toolchain generator or by your build system.
- ▶ Allows to see what any of your processes is doing: accessing files, allocating memory... Often sufficient to find simple bugs.
- ▶ Usage: `strace <command>` (starting a new process) `strace -f <command>` (follow child processes too) `strace -p <pid>` (tracing an existing process) `strace -c <command>` (time statistics per system call) `strace -e <expr> <command>` (use **e**xpression for advanced filtering)

See [the strace manual](#) for details



Image credits: <https://strace.io/>



Measuring: strace

```
> strace cat Makefile
[...]  
fstat64(3, {st_mode=S_IFREG|0644, st_size=111585, ...}) = 0  
mmap2(NULL, 111585, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb7f69000  
close(3) = 0  
access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT (No such file or directory)  
open("/lib/tls/i686/cmov/libc.so.6", O_RDONLY) = 3  
read(3, "\177ELF\1\1\1\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0\320h\1\0004\0\0\0\344"... , 512) = 512  
fstat64(3, {st_mode=S_IFREG|0755, st_size=1442180, ...}) = 0  
mmap2(NULL, 1451632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xb7e06000  
mprotect(0xb7f62000, 4096, PROT_NONE) = 0  
mmap2(0xb7f66000, 9840, PROT_READ|PROT_WRITE,  
      MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xb7f66000  
close(3) = 0  
[...]  
openat(AT_FDCWD, "Makefile", O_RDONLY) = 3  
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=173, ...}, AT_EMPTY_PATH) = 0  
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0  
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7290d28000  
read(3, "ifneq ($(KERNELRELEASE),)\nobj-m "... , 131072) = 173  
write(1, "ifneq ($(KERNELRELEASE),)\nobj-m "... , 173ifneq ($(KERNELRELEASE),)
```

Hint: follow the open file descriptors returned by `open()`. This tells you what files are handled by further system calls.



Measuring: strace

- ▶ Display only a specific set of system calls:

```
$ strace -e 'openat,write' cat Makefile
```

- ▶ Filter out specific system calls:

```
$ strace -e '!poll' cat Makefile
```

- ▶ Show only system calls returning a specific status

```
$ strace -e 'status=failed' cat Makefile
```

- ▶ Trace how a file is accessed and used among different system calls

```
$ strace -P '/etc/ld.so.cache' cat Makefile
```

- ▶ Run `strace -tips` to learn new commands !



Measuring: strace



- ▶ Uses hardware performance counters, much faster than Valgrind!
- ▶ Need a kernel with `CONFIG_PERF_EVENTS` and `CONFIG_HW_PERF_EVENTS`
- ▶ User space tool: `perf`. It is part of the kernel sources so it is always in sync with your kernel.
- ▶ Usage:

```
perf record /my/command
```

- ▶ Get the results with:

```
perf report
```

- ▶ Note: advice to run `perf` on a filesystem built with glibc. Didn't manage to compile `perf` on a Musl root filesystem (Buildroot 2021.02 status). Once again, glibc is recommended for debugging.



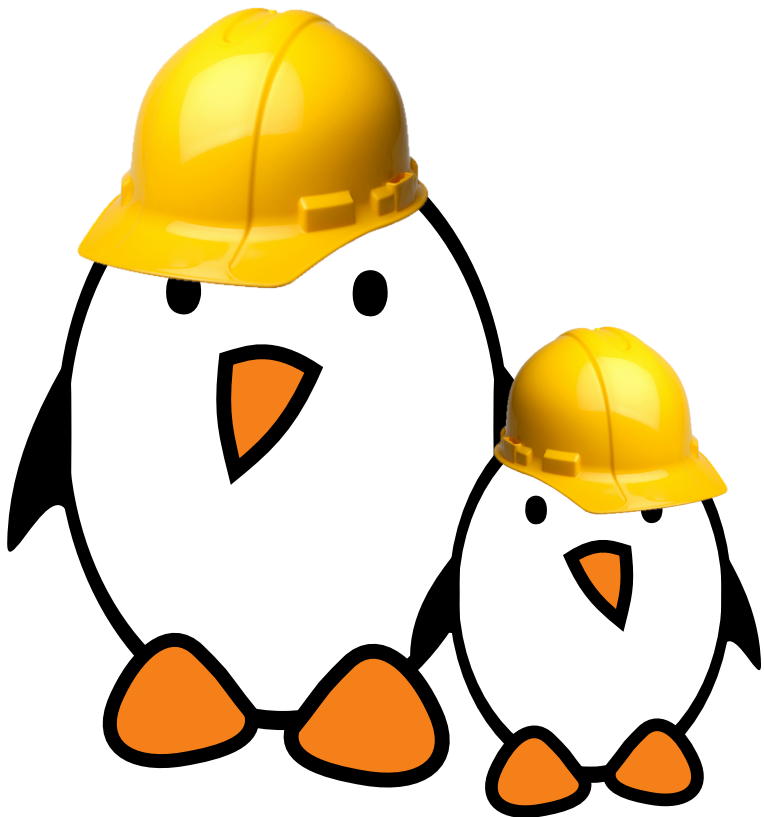
perf report output

```
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 5K of event 'cycles'
# Event count (approx.): 1392529663
#
# Overhead  Command  Shared Object                Symbol
# .....  .....  .....
#
 10.72%   ffmpeg  [kernel.kallsyms]           [k] video_get_user
 10.60%   ffmpeg  [kernel.kallsyms]           [k] vector_swi
  4.76%   ffmpeg  libc-2.31.so                 [.] ioctl
  4.22%   ffmpeg  [kernel.kallsyms]           [k] __se_sys_ioctl
  3.81%   ffmpeg  [kernel.kallsyms]           [k] __video_do_ioctl
  3.42%   ffmpeg  libavformat.so.58.45.100     [.] avformat_find_stream_info
  2.83%   ffmpeg  [kernel.kallsyms]           [k] video_usercopy
  2.70%   ffmpeg  libc-2.31.so                 [.] cfree
  2.58%   ffmpeg  [kernel.kallsyms]           [k] __fget_light
  2.53%   ffmpeg  libpthread-2.31.so           [.] __errno_location
  2.40%   ffmpeg  [kernel.kallsyms]           [k] arm_copy_from_user
  2.26%   ffmpeg  [kernel.kallsyms]           [k] memset
  2.09%   ffmpeg  [kernel.kallsyms]           [k] mutex_unlock
  2.06%   ffmpeg  [kernel.kallsyms]           [k] v4l2_ioctl
  2.05%   ffmpeg  libavcodec.so.58.91.100     [.] av_init_packet
  1.95%   ffmpeg  libc-2.31.so                 [.] memset
...

```



Practical lab - Optimizing the application



- ▶ Compile the video player with just the features needed at run time.
- ▶ Trace and profile the video player with `strace`
- ▶ Observe size and time savings